

Reconstructing Curves Using the Crust

David Skocic

April 9, 2026

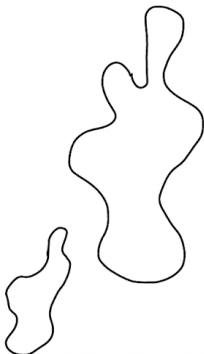
Try for Yourself!



<https://skocicd.github.io/MTH496H/>

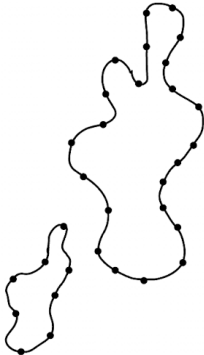
Polygonal Reconstruction of a Curve

- Points are taken from a smooth curve, known as **sampling**
- Segments are drawn to connect all adjacent samples, and connect no other points
- Under the correct sampling conditions, algorithms can guarantee a successful reconstruction



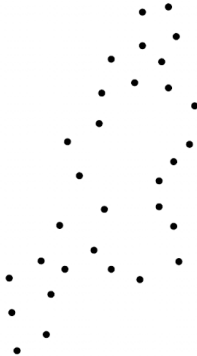
Polygonal Reconstruction of a Curve

- Points are taken from a smooth curve, known as **sampling**
- Segments are drawn to connect all adjacent samples, and connect no other points
- Under the correct sampling conditions, algorithms can guarantee a successful reconstruction



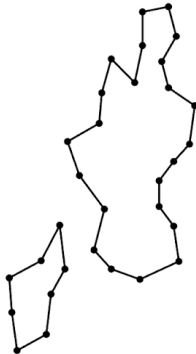
Polygonal Reconstruction of a Curve

- Points are taken from a smooth curve, known as **sampling**
- Segments are drawn to connect all adjacent samples, and connect no other points
- Under the correct sampling conditions, algorithms can guarantee a successful reconstruction



Polygonal Reconstruction of a Curve

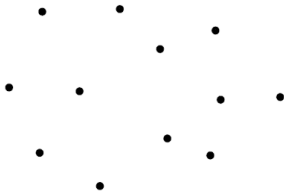
- Points are taken from a smooth curve, known as **sampling**
- Segments are drawn to connect all adjacent samples, and connect no other points
- Under the correct sampling conditions, algorithms can guarantee a successful reconstruction



Geometric Tools

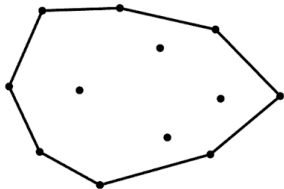
Convex Hull

- The smallest convex shape that contains all the points
- Creates some structure from an unlabeled set of points
- Useful as a starting point for many algorithms



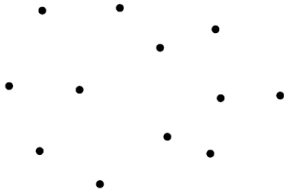
Convex Hull

- The smallest convex shape that contains all the points
- Creates some structure from an unlabeled set of points
- Useful as a starting point for many algorithms



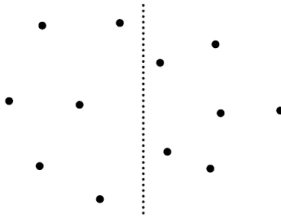
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



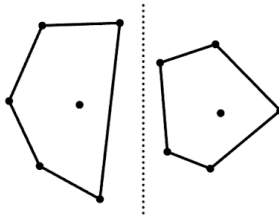
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



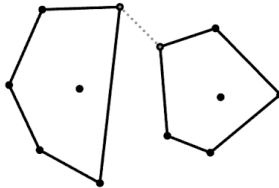
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



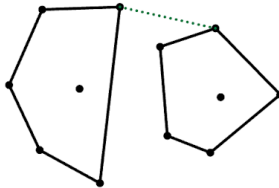
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



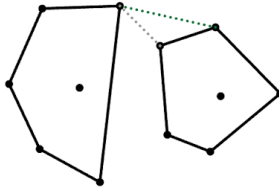
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



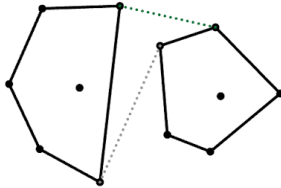
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



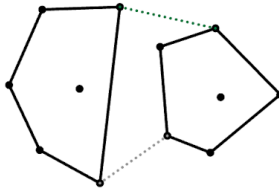
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



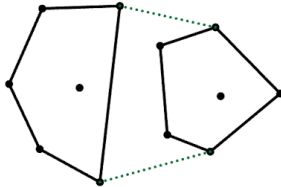
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



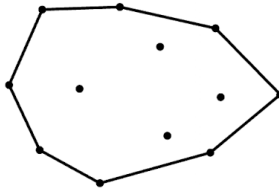
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



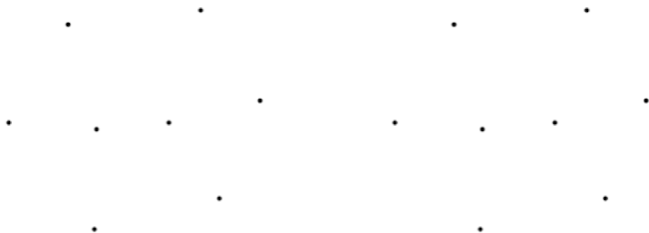
Computing the Convex Hull

- Divide and conquer!
- Assume we can get the convex hull of the left and right half of the points
- Eventually, there will be 2 or 3 points, which are always convex
- Recombine halves by walking segments starting from the innermost points until they are tangent to each hull



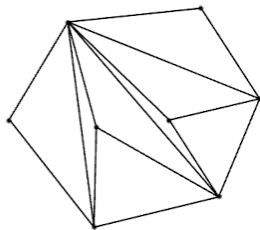
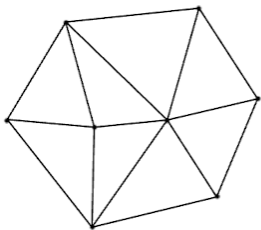
Triangulations

- A triangulation creates a mesh that connects all points in a set using only triangles
- Any set of points can be triangulated, usually in many different ways
- Some triangulations are more useful than others



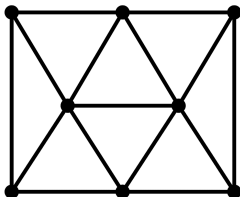
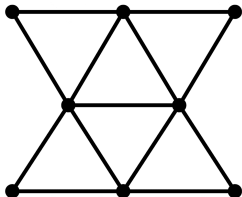
Triangulations

- A triangulation creates a mesh that connects all points in a set using only triangles
- Any set of points can be triangulated, usually in many different ways
- Some triangulations are more useful than others



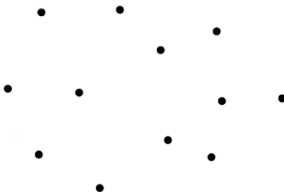
Triangulations

- Any “good” triangulation contains the convex hull
- This ensures that the triangulation has as many triangles as possible
- The convex hull can serve as a starting point to find a triangulation



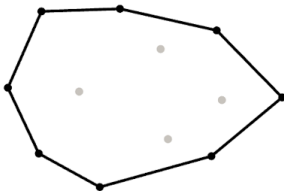
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



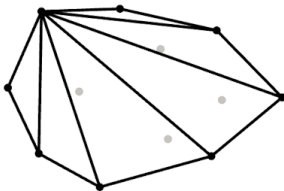
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



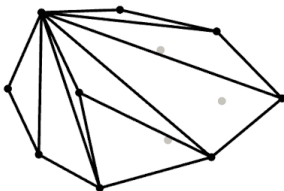
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



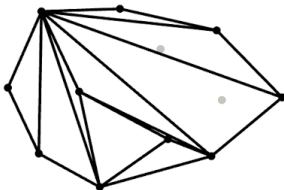
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



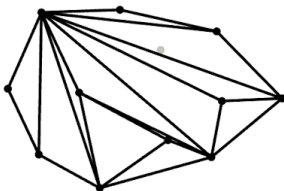
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



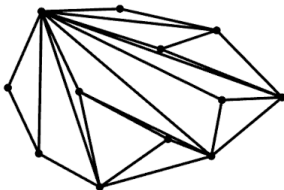
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



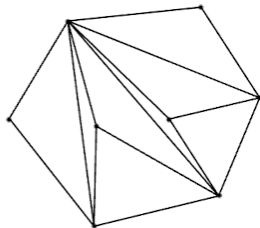
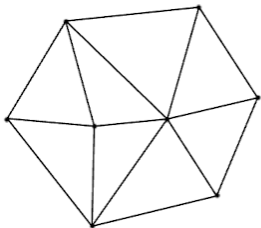
Computing Triangulations

- Start by taking the convex hull
- Connect one vertex to all others in the hull
- Add points one at a time, connecting them with the vertices of the triangle that they are in



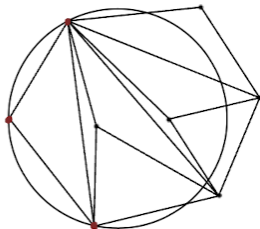
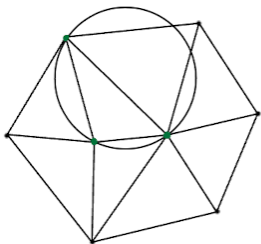
Delaunay Triangulations

- Delaunay triangulations obey the **empty circle property**
- Creates a balanced triangulation, reducing the number of thin triangles
- Always exists and is unique for a given set of points



Delaunay Triangulations

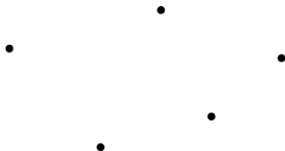
- Delaunay triangulations obey the **empty circle property**
- Creates a balanced triangulation, reducing the number of thin triangles
- Always exists and is unique for a given set of points



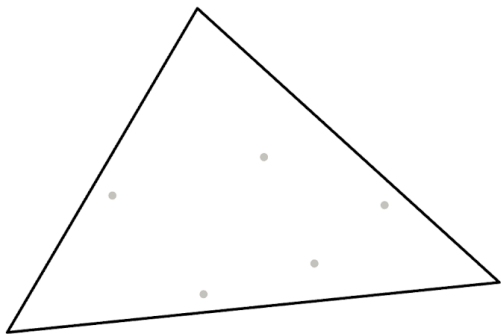
Computing the Delaunay Triangulation

1. Draw a super triangle, large enough to contain every point
2. Add in points one at a time, and find all circumcircles that contain it
3. If two circumcircles come from triangles that share an edge, remove that edge
4. Connect the new point to every vertex surrounding it
5. After all points are added, remove any edge that has a vertex of the super triangle as an endpoint

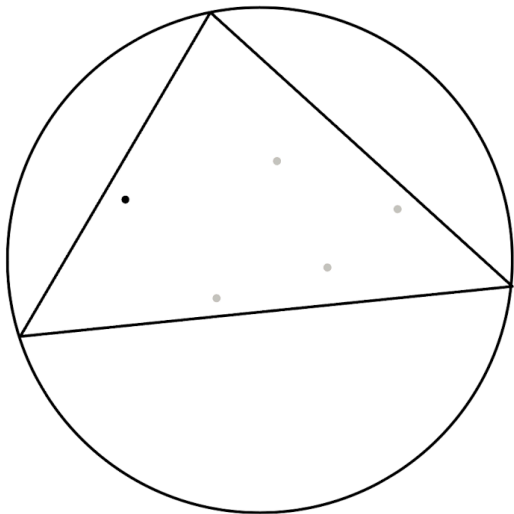
Computing the Delaunay Triangulation



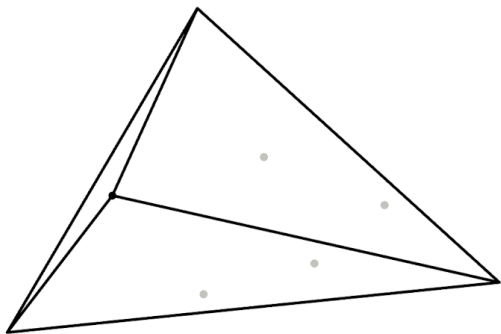
Computing the Delaunay Triangulation



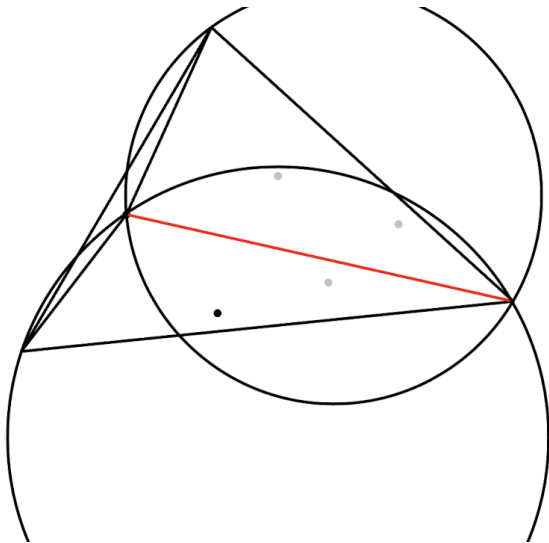
Computing the Delaunay Triangulation



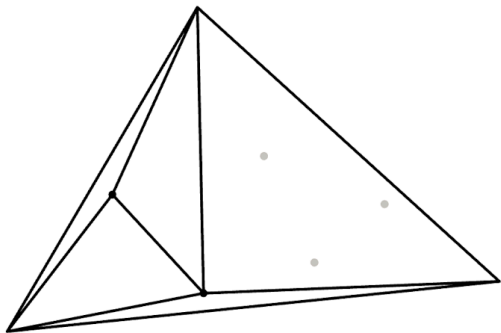
Computing the Delaunay Triangulation



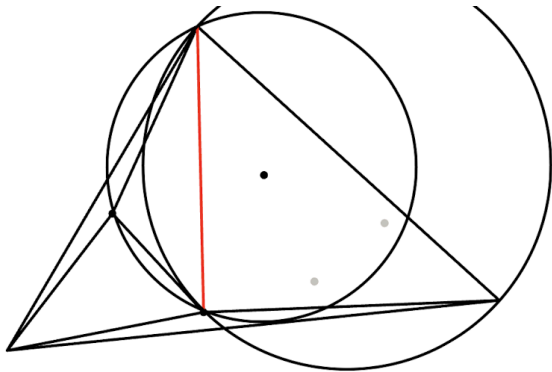
Computing the Delaunay Triangulation



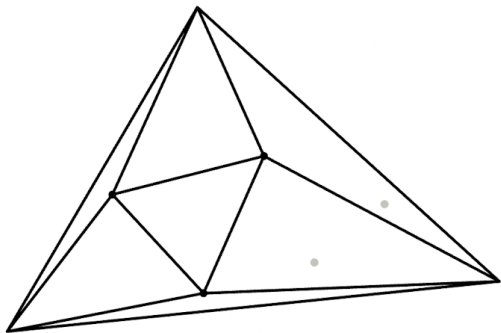
Computing the Delaunay Triangulation



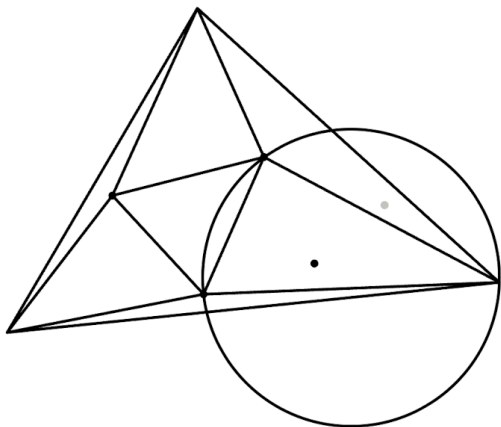
Computing the Delaunay Triangulation



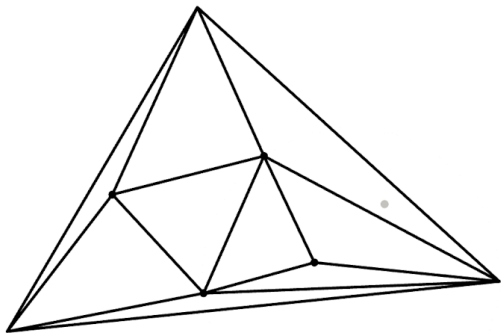
Computing the Delaunay Triangulation



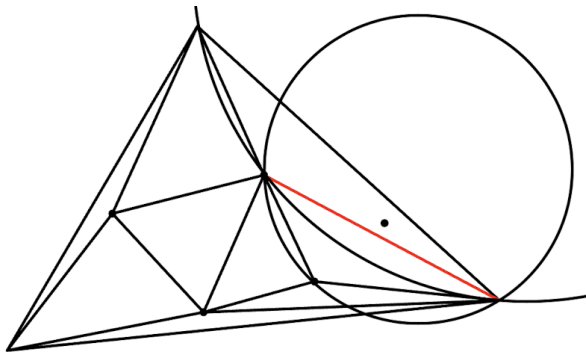
Computing the Delaunay Triangulation



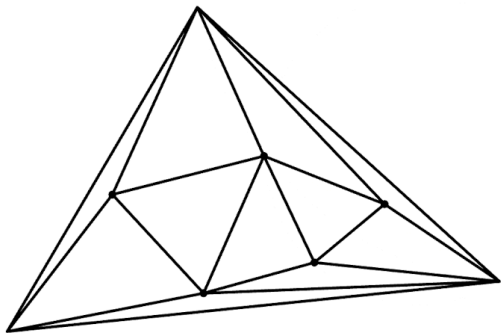
Computing the Delaunay Triangulation



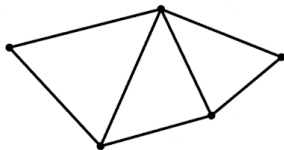
Computing the Delaunay Triangulation



Computing the Delaunay Triangulation

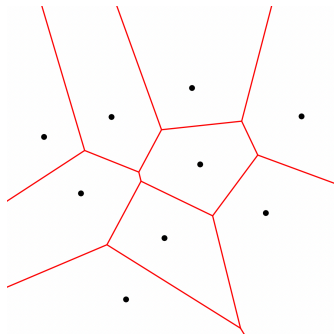


Computing the Delaunay Triangulation



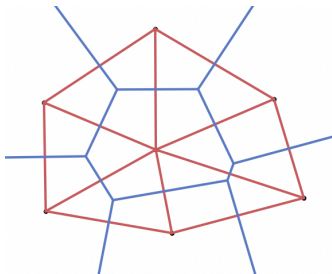
Voronoi Diagrams

- The original set of points are now called **sites**
- Divide the plane into **regions of proximity**
- Consists of Voronoi edges and Voronoi vertices
- Useful to quickly find the nearest site for any given point in the plane



The Delaunay-Voronoi Relationship

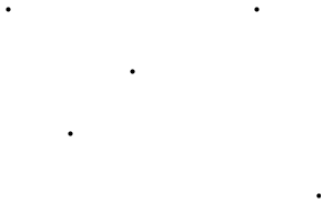
- Voronoi vertices are equidistant to at least 3 sites
- A circle centered at a Voronoi vertex connecting these sites contains no other sites
- Drawing lines between these sites yields a triangle that satisfies the empty circle property
- The Voronoi diagram and the Delaunay Triangulation are **dual graphs** of one another



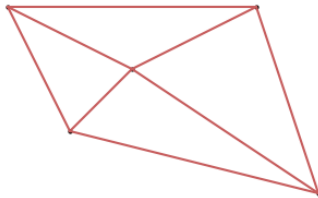
Computing the Voronoi Diagram

1. Start by taking the Delaunay triangulation
2. Find the circumcenter of all triangles, these are the Voronoi vertices
3. Draw lines connecting each pair of Voronoi vertices that come from triangles sharing an edge
4. For any edges on the outside of the triangulation, draw a ray outwards from the Voronoi vertex of its triangle, perpendicular to the edge

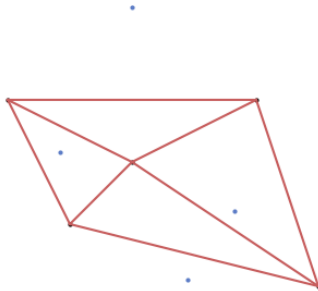
Computing the Voronoi Diagram



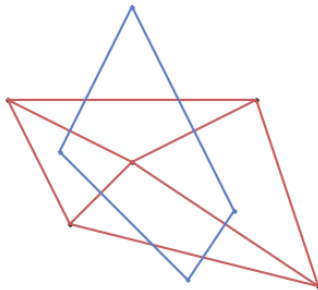
Computing the Voronoi Diagram



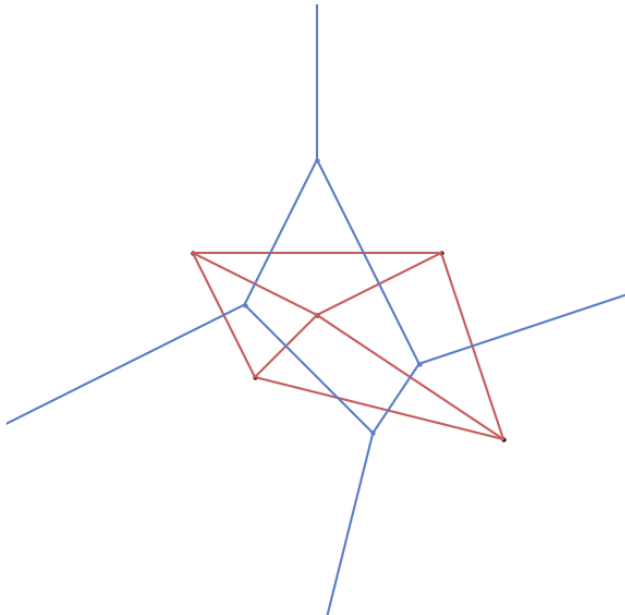
Computing the Voronoi Diagram



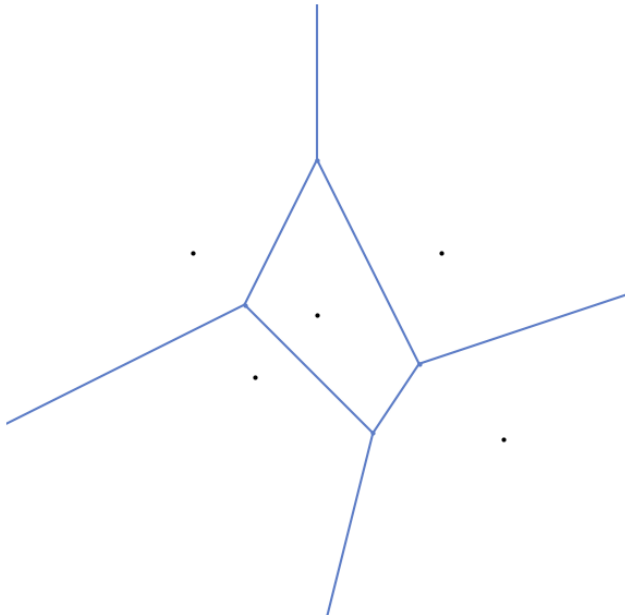
Computing the Voronoi Diagram



Computing the Voronoi Diagram



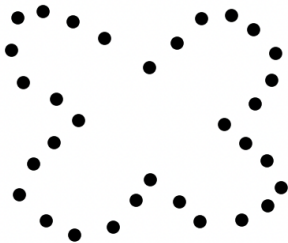
Computing the Voronoi Diagram



The Crust

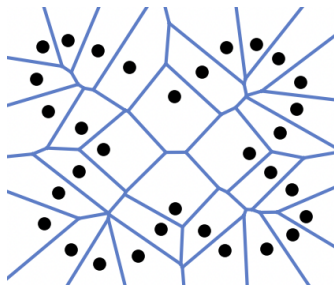
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



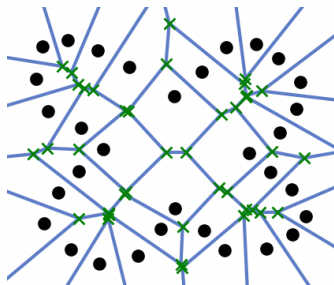
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



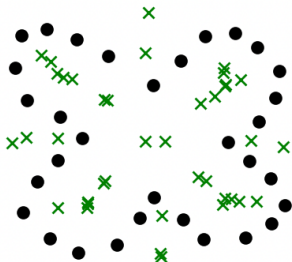
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



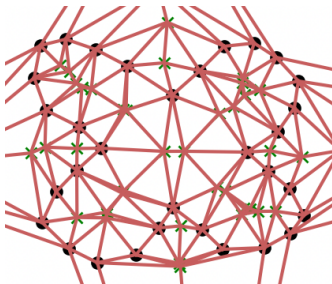
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



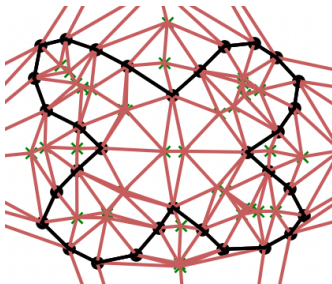
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



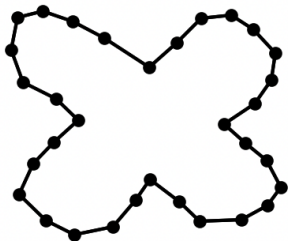
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



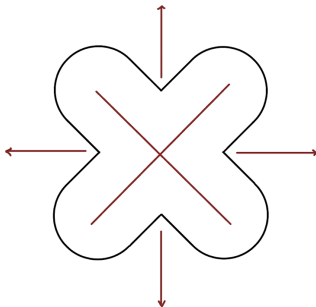
The Crust

1. Start with a set of points S and compute the Voronoi Vertices V .
2. Let $S' = S \cup V$ and let E be the set of edges of the Delaunay triangulation of S'
3. All edges in E that connect two points of S make up the crust



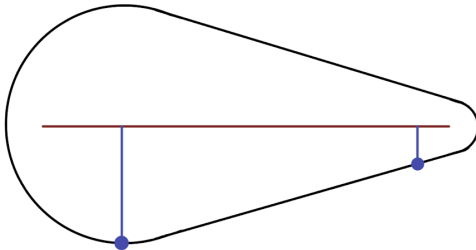
The Medial Axis

- The medial axis is similar to a Voronoi diagram but for an infinite set of points
- The medial axis is the set of all points in the plane that are nearest to more than one point of the smooth curve.
- This definition allows the medial axis to serve as a sort of midline throughout a shape



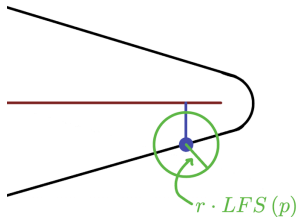
Local Feature Size

- At a point p on a smooth curve, we can define the local feature size, $LFS(p)$ as the distance to the closest point of the medial axis
- As the name suggests, the local feature size gives an idea of how big the features of a shape are at any given point
- When a curve is more detailed, LFS is small, and when a curve is very flat it is large



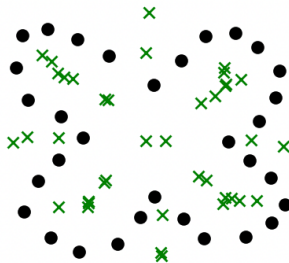
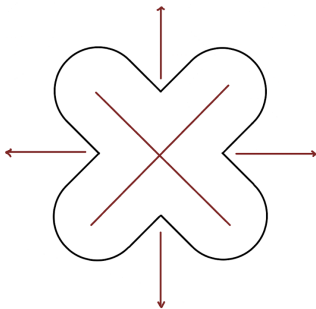
Sampling

- We say that a curve is r -sampled if the distance between any pair of adjacent samples is less than or equal to $r \cdot LFS(p)$
- This means that a more detailed section of a curve has samples very close together, while less detailed sections can be far apart
- An r -sampled curve with $r < .252$ is guaranteed to be successfully reconstructed by the crust



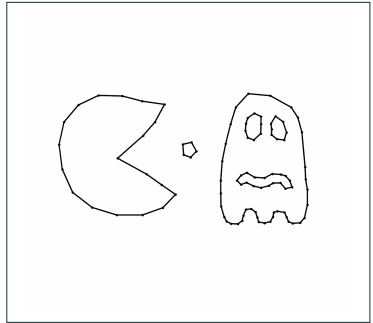
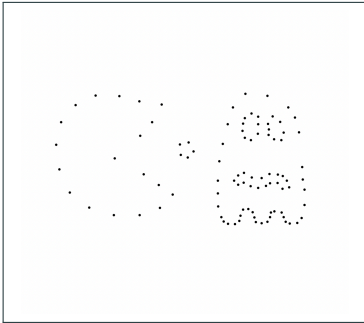
Why Does the Crust Work?

- The Voronoi vertices approximate the medial axis
- A circle connecting two adjacent points in the crust must not contain a Voronoi vertex by the empty circle property
- This means that the crust can't "cut across the middle" of the shape



Conclusion

- The crust uses foundational tools to efficiently reconstruct a curve
- If you can approximate "how detailed" a shape is going to be, you can sample accordingly



Thank you!

Questions?